EF Core Lab 01 - Theoretical Answers

Q1: Why do we use Code First rather than DB First?

Answer:

We prefer **Code First** over **Database First** for several key reasons:

- **Developer-Centric Approach:** Allows developers to design domain models using pure C# classes without needing to context-switch to SQL database design tools.
- **Version Control & Migrations:** Database schema history and changes are tracked directly alongside source code via EF Core Migrations, making deployment and rollback across environments seamless.
- **Object-Oriented Focus:** Prioritizes business logic and domain entities first, letting the database schema adapt to the application rather than forcing code to fit a pre-existing schema.

Q2: Why do we use Fluent API over Data Annotations?

Answer:

We use **Fluent API** over **Data Annotations** because:

- **Separation of Concerns:** Keeps domain entity classes clean and free from database-specific metadata (Pure POCOs).
- **Advanced Configurations:** Supports complex configurations that Data Annotations cannot express (e.g., composite primary keys, shadow properties, advanced cascade deletes, and global query filters).
- **Highest Precedence:** Fluent API overrides both default conventions and Data Annotations, providing absolute control over schema generation.

Q3: What is the difference between `Up()` and `Down()` methods in EF Core Migrations? (`Up to new Change` VS `Up to previous Change`)

Answer: In EF Core Migrations, each generated migration file contains two core methods to manage database version control:

- **`Up()` Method (`Up to new Change`):** Contains the code required to apply new schema changes and upgrade the database forward to the new version.
- **`Down()` Method (`Up to previous Change`):** Contains the code required to revert and undo the changes applied by `Up()`, rolling back the database to its previous state.